

# Operador de Marketing e Inteligencia de Inventario D2C

Manual del Proyecto

Automatización de Marketing y Gestión de Inventario para Retail D2C

Sistema de Agente IA — Arquitectura de 3 Capas

21 de junio de 2026

# Índice

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                      | <b>4</b>  |
| 1.1. Propósito del Sistema . . . . .                                        | 4         |
| 1.2. Stack Tecnológico . . . . .                                            | 4         |
| <b>2. Estructura del Proyecto</b>                                           | <b>4</b>  |
| 2.1. Descripción de Directorios . . . . .                                   | 5         |
| <b>3. Arquitectura del Sistema</b>                                          | <b>5</b>  |
| 3.1. Capa 1: Directivas (directives/) — <i>Qué hacer</i> . . . . .          | 5         |
| 3.2. Capa 2: Orquestación (El Agente) — <i>Toma de decisiones</i> . . . . . | 6         |
| 3.3. Capa 3: Ejecución (execution/) — <i>Hacer el trabajo</i> . . . . .     | 6         |
| <b>4. Configuración Inicial</b>                                             | <b>7</b>  |
| 4.1. Requisitos Previos . . . . .                                           | 7         |
| 4.2. Entorno Virtual . . . . .                                              | 7         |
| 4.3. Docker Sandbox . . . . .                                               | 7         |
| 4.4. Archivo .env . . . . .                                                 | 7         |
| <b>5. Módulos y Funcionalidades</b>                                         | <b>7</b>  |
| 5.1. Gestión de Inventario Shopify . . . . .                                | 7         |
| 5.1.1. Uso . . . . .                                                        | 8         |
| 5.1.2. Parámetros . . . . .                                                 | 8         |
| 5.1.3. Salida . . . . .                                                     | 8         |
| 5.1.4. Manejo de Errores . . . . .                                          | 8         |
| 5.2. Alertas de Inventario y Publicidad . . . . .                           | 8         |
| 5.3. Generación de Contenido de Marketing . . . . .                         | 9         |
| 5.4. Asistente de Soporte al Cliente . . . . .                              | 9         |
| 5.5. Sincronización Whatnot a Shopify . . . . .                             | 9         |
| <b>6. Protocolo de Diagnóstico de Entorno</b>                               | <b>9</b>  |
| <b>7. Memoria Persistente (ChromaDB)</b>                                    | <b>10</b> |
| <b>8. Formato de Salida</b>                                                 | <b>10</b> |
| 8.1. Borradores para Revisión Humana . . . . .                              | 10        |
| 8.2. Alertas Prioritarias . . . . .                                         | 10        |
| 8.3. Estado de Ejecución . . . . .                                          | 10        |
| <b>9. Mantenimiento y Git</b>                                               | <b>10</b> |
| 9.1. Scripts de Actualización . . . . .                                     | 10        |
| 9.1.1. git-update.sh . . . . .                                              | 11        |
| 9.1.2. update_repo.sh . . . . .                                             | 11        |
| <b>10. Principios de Operación</b>                                          | <b>11</b> |
| 10.1. Autocorrección (Self-Annealing) . . . . .                             | 11        |
| 10.2. Reutilización . . . . .                                               | 11        |
| 10.3. Actualización de Directivas . . . . .                                 | 12        |

---

|                                                   |           |
|---------------------------------------------------|-----------|
| 10.4. Validación Continua . . . . .               | 12        |
| <b>11.Seguridad</b>                               | <b>12</b> |
| <b>12.Apéndice: Dependencias</b>                  | <b>12</b> |
| <b>13.Apéndice: Referencia Rápida de Comandos</b> | <b>12</b> |

# 1. Introducción

Este proyecto implementa un **agente de inteligencia artificial** especializado en la automatización de marketing y la gestión de inventario para una tienda **Direct-to-Consumer (D2C)** de ropa de golf. La tienda opera simultáneamente en **Shopify** (tienda online), **Whatnot** (ventas en vivo) y **Meta Ads** (publicidad).

El objetivo central es **reducir las tareas manuales** del equipo humano y optimizar la precisión de las operaciones y el marketing mediante un sistema de alertas, borradores automáticos y sincronización de datos, manteniendo siempre la supervisión humana como paso obligatorio antes de cualquier acción ejecutable.

## 1.1. Propósito del Sistema

- Actuar como puente entre la intención del usuario y la ejecución técnica.
- Separar la lógica probabilística (IA) de la ejecución determinista (scripts).
- Proveer un marco reproducible, con estado trazable y capacidad de autocorrección.

## 1.2. Stack Tecnológico

- **Lenguaje:** Python 3.12+
- **Librerías principales:** ShopifyAPI, python-dotenv, psutil.
- **Infraestructura:** Docker (sandbox para compilación nativa), Conda (gestión de entornos).
- **Automatización:** Compatible con Zapier / Make.com.
- **Memoria persistente:** ChromaDB.
- **SO base:** Linux (Kernel compatible con Linux Mint).

# 2. Estructura del Proyecto

La siguiente es la estructura de directorios del proyecto:

```

1  .
2  +-- .agent/                                # Instrucciones de sistema y
    contexto del agente
3  |   +-- AGENT_FRAMEWORK.md                # Arquitectura de 3 capas y setup
4  |   +-- AGENT_INSTRUCTIONS.md             # Protocolo de agente y memoria
    evolutiva
5  |   +-- Dockerfile                        # Imagen para sandbox Docker
6  +-- .env                                  # Credenciales y configuracion del
    agente
7  +-- .gitignore
8  +-- directives/                          # SOPs en YAML (Layer 1)
9  |   +-- check_shopify_inventory.yaml
10 +-- docs/

```

```

11 |   +-- OBJETIVO.md           # Prompt principal del agente
12 +-- execution/              # Scripts deterministas en Python
   (Layer 3)
13 |   +-- env_diagnostic.py
14 |   +-- shopify_get_inventory.py
15 +-- requirements.txt        # Dependencias del proyecto
16 +-- git-update.sh           # Script de actualizacion Git
17 +-- update_repo.sh          # Script de sincronizacion de
   repositorio
18 +-- env_diagnostic.py        # Diagnostico de entorno
19 +-- README.md
20 +-- manual.tex               # Este documento

```

## 2.1. Descripción de Directorios

**.agent/** Contiene las instrucciones del sistema y la definición del marco de trabajo del agente (arquitectura de 3 capas, protocolos de ejecución, memoria).

**directives/** Archivos YAML que actúan como Procedimientos Operativos Estandarizados (SOP). Cada flujo de trabajo repetible tiene su propia directiva.

**execution/** Scripts Python deterministas con una única responsabilidad. Son invocados por la capa de orquestación.

**.tmp/** Archivos temporales y estado de ejecución (regenerables).

**docs/** Documentación del proyecto.

## 3. Arquitectura del Sistema

El sistema se basa en una **arquitectura de 3 capas** diseñada para maximizar la fiabilidad, separando la lógica probabilística del LLM de la ejecución determinista.

### 3.1. Capa 1: Directivas (directives/) — *Qué hacer*

Son SOPs en formato YAML, con un archivo por flujo de trabajo repetible. Cada directiva contiene:

- **Goal:** Objetivo principal del flujo.
- **Required inputs:** Lista de datos necesarios (nombre y descripción).
- **Steps:** Secuencia de pasos que invocan scripts de execution/.
- **Expected outputs:** Resultados esperados.
- **Edge cases:** Casos límite y protocolos de recuperación.

Ejemplo de estructura de una directiva:

```
1 goal: "Obtener niveles de stock actuales de Shopify"
2 required_inputs:
3   - name: "inventory_threshold"
4     description: "Stock minimo antes de alertar (ej. 5)"
5 steps:
6   - step: 1
7     description: "Consultar API de Shopify"
8     script: "execution/shopify_get_inventory.py"
9 expected_outputs:
10  - "Archivo JSON en .tmp/inventory_report.json"
11 edge_cases:
12  - case: "Error de conexion"
13    protocol: "Reintentar hasta 3 veces"
```

### 3.2. Capa 2: Orquestación (El Agente) — *Toma de decisiones*

Esta capa es el **orquestador inteligente**. Sus funciones incluyen:

- Leer la directiva apropiada para la tarea solicitada.
- Elegir las herramientas de execution/.
- Ejecutar flujos multietapa.
- Validar entradas y salidas.
- Gestionar errores y autocorrección (self-annealing).
- Guardar estado en .tmp/run\_state.json.
- Actualizar directivas con conocimiento aprendido.

**Regla fundamental:** No inventar lógica que deba vivir en los scripts. El agente conecta intención e implementación.

### 3.3. Capa 3: Ejecución (execution/) — *Hacer el trabajo*

Scripts Python deterministas con los siguientes requisitos:

- Entradas por argumentos de línea de comandos.
- Secretos en .env (nunca hardcodeados).
- Salidas por stdout (preferiblemente en JSON).
- Códigos de salida: 0 = éxito, 1+ = fallos categorizados.
- Validación propia de salidas.
- Fallo ruidoso ante condiciones anómalas.

## 4. Configuración Inicial

### 4.1. Requisitos Previos

- Python 3.12 o superior.
- Conda (recomendado) o virtualenv.
- Docker (para sandbox de compilación).
- Acceso a Internet y a GitHub.

### 4.2. Entorno Virtual

```
1 # Crear entorno Conda
2 conda create --name d2c_env python=3.12 -y
3
4 # Activar entorno
5 conda activate d2c_env
6
7 # Instalar dependencias
8 pip install -r requirements.txt
```

### 4.3. Docker Sandbox

Para ejecución de código C/C++ de forma aislada:

```
1 docker build -t pcb_sandbox .agent/
```

### 4.4. Archivo .env

El archivo .env contiene las credenciales necesarias:

- SHOPIFY\_SHOP\_URL — URL de la tienda Shopify.
- SHOPIFY\_ACCESS\_TOKEN — Token de acceso a la API de Shopify.
- Claves de API para LLMs (Google, OpenAI, Anthropic, etc.).
- Configuración de Telegram (bot token, chat ID).
- Configuración de gestión de contexto (límites de mensajes y caracteres).

## 5. Módulos y Funcionalidades

### 5.1. Gestión de Inventario Shopify

El script `execution/shopify_get_inventory.py` consulta la API de Shopify y genera un reporte de inventario.

### 5.1.1. Uso

```
1 python execution/shopify_get_inventory.py --threshold 5
```

### 5.1.2. Parámetros

—**threshold** Nivel de stock por debajo del cual se considera “bajo inventario” (por defecto: 5).

### 5.1.3. Salida

Genera un archivo JSON en `.tmp/inventory_report.json` con la siguiente estructura:

```
1 {
2   "out_of_stock": [
3     { "id": 123, "title": "Producto - Variante",
4       "sku": "GOLF-001", "stock": 0 }
5   ],
6   "low_stock": [
7     { "id": 456, "title": "Producto2 - Variante2",
8       "sku": "GOLF-002", "stock": 3 }
9   ],
10  "summary": { "total_checked": 150 }
11 }
```

### 5.1.4. Manejo de Errores

- **Error de conexión:** Reintentar hasta 3 veces.
- **API Rate Limit:** Detectar header `Retry-After` y pausar automáticamente.
- **Credenciales faltantes:** Mensaje de error claro en JSON.

## 5.2. Alertas de Inventario y Publicidad

El sistema cruza el gasto en anuncios de Meta Ads contra el stock disponible en Shopify para:

- Identificar anuncios activos de productos agotados o con bajo stock.
- Generar **Alertas Prioritarias** cuando hay discrepancia entre gasto publicitario y stock.
- Detectar productos recién repuestos para activar campañas de marketing.



### 5.3. Generación de Contenido de Marketing

Basado en la disponibilidad de inventario, el agente genera borradores de:

- Campañas de Email (flujos Klaviyo).
- Anuncios para Meta (Facebook/Instagram).
- Contenido para redes sociales.
- SMS marketing.

Todo el contenido se alinea con la identidad de la marca de ropa de golf y está diseñado para ser atractivo para la comunidad del golf.

### 5.4. Asistente de Soporte al Cliente

El agente redacta borradores de respuesta a consultas de soporte extrayendo información de:

- Pedidos de Shopify (estado, productos, fechas).
- Políticas de la empresa (devoluciones, envíos).
- Historial de respuestas anteriores.

El usuario solo debe revisar y enviar.

### 5.5. Sincronización Whatnot a Shopify

El sistema ayuda a estructurar la lógica para:

- Mapear ventas realizadas en vivo en Whatnot hacia pedidos en Shopify.
- Asegurar que los datos de inventario estén listos para imprimir hojas de stock para shows en vivo.

## 6. Protocolo de Diagnóstico de Entorno

Antes de cualquier ejecución, el agente debe conocer el entorno real del sistema. El protocolo es:

1. Generar `env_diagnostic.py` que recolecte: SO, arquitectura, versión de Python, entorno Conda, paquetes críticos, RAM, CPU, GPU, permisos, conectividad.
2. El usuario ejecuta el script y pega la salida.
3. El agente ajusta su ejecución a las versiones y limitaciones confirmadas.
4. Si cambia el contexto de trabajo o surge un `ImportError`, solicitar nuevo diagnóstico.

## 7. Memoria Persistente (ChromaDB)

El agente utiliza ChromaDB para memoria persistente:

- **Consulta inicial:** Antes de proponer una solución, consulta la memoria para experiencias pasadas.
- **Registro de aprendizaje:** Errores críticos y limitaciones técnicas se guardan con `save_memory.yaml`.
- **Autocorrección:** Si un script falla, busca fallos similares antes de intentar una nueva solución.

Directivas disponibles:

- `save_memory.yaml` — Guardar aprendizaje en memoria.
- `query_memory.yaml` — Recuperar información relevante.
- `list_memories.yaml` — Listar recuerdos recientes.
- `delete_memory.yaml` — Eliminar un recuerdo por ID.

## 8. Formato de Salida

### 8.1. Borradores para Revisión Humana

Todo contenido generado por el agente (emails, publicaciones, respuestas de soporte) se presenta como **borrador**. El usuario mantiene el control final y debe aprobar antes de ejecutar.

### 8.2. Alertas Prioritarias

Cuando se detecta una discrepancia entre el gasto publicitario y el stock disponible, se genera una **Alerta Prioritaria** que se resalta en la salida para acción inmediata.

### 8.3. Estado de Ejecución

Cada paso de ejecución se registra en `.tmp/run_state.json` con:

- Timestamp.
- Script ejecutado.
- Código de salida (`exit_code`).

## 9. Mantenimiento y Git

### 9.1. Scripts de Actualización

El proyecto incluye dos scripts para la gestión de versiones:

### 9.1.1. git-update.sh

Script principal que:

1. Detecta la rama actual.
2. Commitea cambios locales con mensaje “WIP: guardar cambios antes de pull”.
3. Delega en `update_repo.sh`.

### 9.1.2. update\_repo.sh

Script de sincronización con soporte para:

```

1 ./update_repo.sh                # Pull + commit + push
2 ./update_repo.sh --no-pull --push # Solo commit y push
3 ./update_repo.sh --dry-run       # Vista previa sin ejecutar
4 ./update_repo.sh --confirm       # Modo interactivo con
   confirmacion
5 ./update_repo.sh -m "Mensaje"    # Mensaje de commit
   personalizado

```

**Parámetros:**

**--dir DIR** Directorio del repositorio (por defecto: actual).

**--remote REMOTE** Remote Git (por defecto: origin).

**--branch BRANCH** Rama (por defecto: actual).

**--push** Hacer push después del commit.

**--no-pull** Omitir pull inicial.

**--dry-run** Vista previa sin cambios.

**--confirm** Modo interactivo.

## 10. Principios de Operación

### 10.1. Autocorrección (Self-Annealing)

- Presupuesto máximo de 3 reintentos por tarea.
- Clasificar fallos en: **Lógica**, **Entorno** o **Recursos**.
- Analizar causa raíz antes de corregir.
- Fiabilidad sobre velocidad: mejor preguntar que continuar con datos inconsistentes.

### 10.2. Reutilización

Antes de escribir un nuevo script, verificar si existe en `execution/` una herramienta que pueda reutilizarse.

### 10.3. Actualización de Directivas

Cada hallazgo (errores comunes, límites, mejoras) debe registrarse en las directivas sin sobrescribir versiones anteriores.

### 10.4. Validación Continua

Después de cada paso, confirmar:

- Esquema correcto.
- Cantidades coherentes.
- Archivos en las rutas esperadas.
- Tiempos y fechas razonables.

## 11. Seguridad

- **Sanitización de entradas:** Verificar que rutas y parámetros no contengan caracteres de escape maliciosos (;, &, |, etc.).
- **Validación de tipos:** Los scripts deben forzar tipos de datos (Type Hinting) para evitar errores de casting en runtime.
- **Credenciales:** Nunca hardcodear secretos en los scripts; usar `.env`.
- **Aislamiento:** Uso de Docker para compilación de código no confiable.

## 12. Apéndice: Dependencias

El archivo `requirements.txt` contiene:

```
1 psutil
2 python-dotenv
3 ShopifyAPI
```

## 13. Apéndice: Referencia Rápida de Comandos

| Comando                                                             | Descripción                   |
|---------------------------------------------------------------------|-------------------------------|
| <code>conda create -n d2c_env python=3.12 -y</code>                 | Crear entorno                 |
| <code>conda activate d2c_env</code>                                 | Activar entorno               |
| <code>pip install -r requirements.txt</code>                        | Instalar dependencias         |
| <code>python execution/shopify_get_inventory.py -threshold 5</code> | Consultar inventario          |
| <code>./git-update.sh</code>                                        | Actualizar repositorio        |
| <code>./update_repo.sh -dry-run</code>                              | Vista previa de actualización |
| <code>docker build -t pcb_sandbox .agent/</code>                    | Construir sandbox Docker      |